



Legacy Portal Frontend

Final Technical Documentation

Version	1.0
Prepared for	IG-Techonology
Prepared by	saad farooq
System	Legacy Portal Frontend (Next.js)

Single-source handover document for architecture, API integration, access control, subscription gating, and QA/UAT checklist.

1. Architecture Overview

Stack

Concern	Choice
Framework	Next.js <code>16.0.10</code> (App Router, <code>app/</code> directory)
UI runtime	React <code>19.2</code>
Language	TypeScript <code>5</code>
Styling	Tailwind CSS <code>4</code> + <code>tailwindcss-animate</code> / <code>tw-animate-css</code>
Components	Radix UI primitives wrapped in <code>components/ui/*</code> (shadcn-style)
Data fetching	SWR <code>2.3.8</code>
Charts	Recharts <code>2.15.4</code>
Toasts	Sonner
Icons	lucide-react / @tabler/icons-react
Theming	next-themes (<code>light</code> , <code>dark</code> , <code>client</code>)
PDF export	jsPDF + jsPDF-autotable + svg2pdf / html2canvas
Auth tokens	Custom REST wrapper over Google Identity Toolkit (no client Firebase Auth SDK session)

Two backends (important)

The frontend talks to **two** different origins:

1. **Same-origin Next.js route handlers** (`app/api/auth/*`) — a thin proxy for authentication so the Firebase/Identity Toolkit API key stays server-side. Routes:
 - `POST /api/auth/sign-in`

- `POST /api/auth/sign-up`
- `POST /api/auth/refresh`
- `POST /api/auth/update-password`
- `GET /api/dev/firebase-env` (development diagnostics only)

2. **Cloud Run business API** — all data/business endpoints (gateways, devices, telemetry, users, clinic users, analytics, subscription, reports). Base URL is hardcoded in `lib/portal-config.ts` :
`https://legacy-portal-api-test-601431816450.europe-west2.run.app`

Request/auth flow

```

Browser
  | email/password
  ▼
/api/auth/sign-in (Next route) → Google Identity Toolkit (server-side, key
hidden)
  | { idToken, refreshToken }
  ▼
AuthProvider (contexts/auth-context.tsx)
  - persists refreshToken in sessionStorage
  - decodes JWT claims (clinicId, role)
  - exposes getToken() to the API client
  ▼
lib/api/client.ts → Cloud Run API (Authorization: Bearer <idToken>)
  - on 401: refresh token once, retry, else redirect /login
  - on 402 / payment_required: dispatch paypal event
  - on 403 provisioning codes: redirect to /account/*
  - reads X-Subscription-Warning: past_due header → past-due banner event

```

- ID tokens auto-refresh ~2 minutes before expiry (`RestAuthUser.getIdToken`).
- Parallel refreshes are coalesced (`refreshInFlight` map) to avoid token stampede on first paint.

Trial mode

`TrialProvider` (`contexts/trial-context.tsx`) calls `GET <api>/health` on boot. If the backend reports `trialMode: true`, the app runs without Bearer auth against a fixed trial clinic, and subscription/clinic gating is relaxed. Controlled at build time by `NEXT_PUBLIC_TRIAL_MODE` as a fallback when `/health` is unreachable.

Provider tree (`app/layout.tsx` → `components/providers.tsx`)

```
ThemeProvider
└─ TrialProvider
   └─ AuthProvider
      └─ AutoRefreshSWRConfig      (global SWR refresh interval from settings)
         └─ LatestReadingProvider
            └─ AppLayout           (shell, route guards, paywall)
```

2. Module Map

Routes (`app/`)

Route	Purpose	Notes / access
<code>/</code>	Dashboard overview	authenticated
<code>/analytics</code>	Analytics dashboard	authenticated
<code>/gateways</code> , <code>/gateways/[id]</code>	Gateway list + detail	authenticated
<code>/devices</code> , <code>/devices/[id]</code>	Device list + detail	authenticated
<code>/devices/compare</code>	Side-by-side device comparison	authenticated
<code>/compliance</code>	Compliance view	authenticated
<code>/latest-reading</code>	Latest readings view	authenticated
<code>/users</code> , <code>/users/[id]</code>	Clinic users directory (read)	hidden for doctor/nurse/readonly
<code>/people</code>	Staff management	admin/super_admin (non-personal)
<code>/email-automation</code>	Manual report send + schedules	admin/super_admin (+ doctor for schedules)
<code>/payment</code>	Billing & subscription	admin/super_admin only
<code>/payment/success</code> , <code>/payment/cancelled</code>	Stripe redirect landing pages	shell-less
<code>/settings</code>	Auto-refresh + preferences	authenticated
<code>/login</code> , <code>/signup</code> , <code>/forgot-password</code>	Auth screens	public
<code>/account/pending-clinic</code>	Provisioning waiting state	redirected by API 403 codes
<code>/alerts</code> , <code>/payments</code>	Secondary views	authenticated

Key libraries (`lib/`)

File	Responsibility
<code>lib/api/client.ts</code>	Single API client: all Cloud Run calls, auth header injection, 401 retry, error interceptors, SWR fetchers.
<code>lib/api/types.ts</code>	All request/response TypeScript types.
<code>lib/api/api-error.ts</code>	<code>ApiError</code> , <code>API_ERROR_CODES</code> , <code>isApiError</code> .
<code>lib/portal-config.ts</code>	Hardcoded Cloud Run base URL + Firebase web config.
<code>lib/identity-toolkit-rest.ts</code>	Client-side auth REST wrapper, token persistence.
<code>lib/identity-toolkit-core.ts</code>	Server-side Identity Toolkit calls.
<code>lib/auto-refresh-settings.ts</code>	Reads/writes SWR auto-refresh prefs.
<code>lib/serial-number.ts</code>	Serial number formatting helpers.
<code>lib/device-status.ts</code> ...	Domain formatting/derivation.
<code>lib/pdf/*</code>	PDF report generation.

Contexts & Components

- `contexts/auth-context.tsx` : `user` , `userInfo` , `signIn/signUp/signOut` , `getToken` , `subscriptionStatus` , `isSuperAdmin` .
- `contexts/trial-context.tsx` : `trialMode` , `loading` .
- `contexts/latest-reading-context.tsx` : Shared latest-reading state.
- `components/layout/` : `app-layout` (shell + guards + payroll), `sidebar` , `header` , `footer` .
- `components/ui/` : Shared design-system primitives (~90 components).

3. API Integration Guide

All business calls are centralized in `lib/api/client.ts` .

Standard envelope:

```
// success
{ "success": true, "data": <T>, "cursor": { "next": "<token>" } }

// error
{ "success": false, "error": { "code": "<code>", "message": "<text>" } }
```

Client primitives

Export	Use
<code>fetchApi<T>(path, options?)</code>	Authenticated JSON request; throws <code>ApiError</code> on non-2xx
<code>fetcher(url)</code>	SWR fetcher (unwraps <code>data</code>)
<code>packetFetcher</code> , <code>cursorFetcher</code>	SWR fetchers for paginated responses
<code>setAuthTokenGetter</code> , <code>setFallbackIdToken</code>	Wired by <code>AuthProvider</code> to attach <code>Authorization</code>
<code>isApiError</code> , <code>API_ERROR_CODES</code>	Error narrowing + known backend codes

Endpoint usage by module

Authentication proxy (<code>app/api/auth/*</code>)		
<code>signInWithPassword</code>	POST	<code>/api/auth/sign-in</code>
<code>signUp</code>	POST	<code>/api/auth/sign-up</code>
<code>refreshIdToken</code>	POST	<code>/api/auth/refresh</code>
Session/profile		
<code>getAuthMe</code>	GET	<code>/api/auth/me</code>
<code>claimClinic(code)</code>	POST	<code>/api/auth/claim-clinic</code>
Gateways & Devices		
<code>getGateways</code>	GET	<code>/api/gateways</code>
<code>getDevices</code>	GET	<code>/api/devices</code>
<code>getDeviceCompare(...)</code>	GET	<code>/api/devices/compare?left&right&days</code>
Users / clinic staff		
<code>getUsers</code>	GET	<code>/api/users*</code>
<code>createClinicUser</code>	POST	<code>/api/clinic/users</code>
Reports / scheduler		
<code>sendReportEmail</code>	POST	<code>/api/reports/email/send</code>
<code>listReportSchedules</code>	GET	<code>/api/reports/email-automation/schedules</code>
Billing / subscription		
<code>getSubscriptionStatus</code>	GET	<code>/api/subscription/status</code>
<code>createSubscriptionCheckout</code>	POST	<code>/api/subscription/create-checkout</code>

4. Access Control & Subscription

UI visibility matrix

Module / action	super_admin	admin	doctor	nurse	readonly
Dashboard / devices / gateways	✓	✓	✓	✓	✓
Users directory (/users)	✓	✓	✗	✗	✗
Staff management (/people)	✓	✓*	✗	✗	✗
Email automation manual send	✓	✓	✗	✗	✗
Email automation schedules	✓	✓	✓	✗	✗
Billing (/payment)	✓	✓	✗	✗	✗

* admin requires non-personal clinic account.

Subscription gating flow

Backend contract for protected routes:

- `active` → allow
- `past_due` → allow + `X-Subscription-Warning: past_due`
- `unpaid` / `cancelled` / `unknown` → `402` + `error.code=payment_required`

Frontend reaction:

- `lib/api/client.ts` detects `402`, sets `sessionStorage["legacy-payment-required"] = "1"`, dispatches event.
- `components/layout/app-layout.tsx` listens, blurs/locks shell and shows `PaymentRequiredModal`.

Error-handling matrix

Status	Code	Behavior
400	bad_request	validation/form hint
401	unauthorized	refresh once then <code>/login</code>
401	otp_required	keep on login, show OTP flow
402	payment_required	paywall + lock protected shell
403	forbidden	permission denied UX
403	pending_clinic	redirect <code>/account/*</code>
500/502/503	internal	generic safe message

5. Environment Setup

Prerequisites: Node.js `>=20`, npm

```
npm install
npm run dev
```

Config flags

Variable	Effect
<code>NEXT_PUBLIC_TRIAL_MODE</code>	Fallback trial behavior when <code>/health</code> unavailable
<code>NEXT_PUBLIC_PAYMENT_DEV_PREVIEW</code>	Render payment page without real auth in preview/dev
<code>NEXT_PUBLIC MOCK_STRIPE_CHECKOUT</code>	Mock checkout fallback to <code>/payment/success?mock=1</code>
<code>NEXT_PUBLIC_ENABLE_DUMMY_LOGIN</code>	Enables demo login shortcut

6. Deployment Notes

Deployed to **Google Cloud Run** via `./deploy.sh`.

- Service: `legacy-dashboard`, region `europe-west2`, port `8080`.
- `--allow-unauthenticated` (public web app; API auth is enforced via Bearer token at the API).
- Memory `512Mi`, CPU `1`.
- Build is a standard `next build`. No server-side secret required for the web tier beyond what `app/api/auth/*` needs to reach Identity Toolkit.

7. Known Limitations / Risks

- **Hardcoded API base URL.** Switching environments requires editing `lib/portal-config.ts`.
- **Subscription enforcement is backend-driven.** The frontend can only blur/lock screens when the API returns `402`.
- **Non-admin subscription status.** Non-admins never fetch subscription status directly and rely solely on `402` signals.
- **Trial mode bypass.** If `/health` returns `trialMode: true`, auth and gating are relaxed by design.
- **Staff `/people` vs `/users`.** `/users` is read-only; `/people` is the management surface.
- **Refresh token in `sessionStorage`.** Sessions do not persist across browser restarts / new tabs by design.

8. QA / UAT Checklist

Pre-flight & Auth

- ☐ Confirm backend URL in `lib/portal-config.ts`.
- ☐ Check `<api>/health` and note `trialMode`.
- ☐ Valid login succeeds, invalid login shows safe error.
- ☐ OTP flow works (required, invalid, resend).
- ☐ Signout redirects to `/login`.

Subscription gating & Roles

- ☐ Unpaid user sees blur + paywall; cannot interact with protected screens.
- ☐ Active user and `super_admin` have normal access.

- ☐ past_due only shows soft banner.
- ☐ doctor/nurse/readonly do not see Users and Billing.
- ☐ Non-admin direct `/payment` access redirects away.

Device compare & Reports

- ☐ Compare loads via single `/api/devices/compare` flow.
- ☐ Create/update/delete automation schedule works.
- ☐ Validation errors shown for bad time/timezone/day fields.
- ☐ Manual send validates date range/recipients and handles success/errors.

Billing (admin/super_admin)

- ☐ Billing overview/payment method/invoices load.
- ☐ Checkout, cancel, resume, portal session, change-plan work.